

Deep Learning on VOXL 2's GPU and NPU

Master Pipeline

passed

(https://gitlab.com/voxl-public/voxl-sdk/services/voxl-tflite-server/-/commits/master)

Dev Pipeline

passed

(https://gitlab.com/voxl-public/voxl-sdk/services/voxl-tflite-server/-/commits/dev)

Latest Release

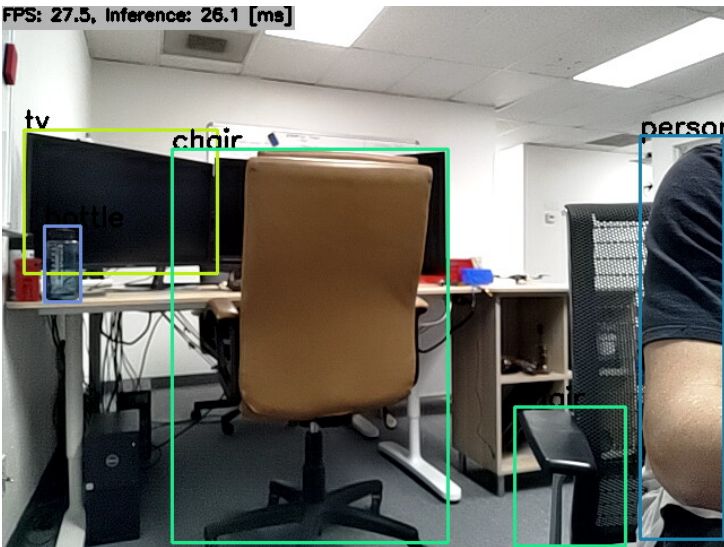
v0.5.1

(https://gitlab.com/voxl-public/voxl-sdk/services/voxl-tflite-server/-/releases)

TABLE OF CONTENTS

- 1 Configuration
- 2 Output
- 3 Benchmarks
- 4 Custom Models
 - a 1. Convert your model to LiteRT
 - b 2. Deploy — no code changes
 - c 3. Only for genuinely new output layouts — extend the server
- 5 Running Multiple Models
- 6 Current Limitations
- 7 Troubleshooting
- 8 Demo
- 9 Source
- 10 Frequently Asked Questions (FAQ)

VOXL 2 delivers state-of-the-art machine learning performance through both GPU and NPU acceleration on the QRB5165. This is best illustrated through `voxl-tflite-server`, our service for on-device inference of LiteRT (<https://ai.google.dev/edge/litert>) (formerly TensorFlow Lite) models. Current releases build against LiteRT/tflite 2.17.1 and target QRB5165-based platforms (VOXL 2 and VOXL 2 Mini); the original VOXL 1 is no longer supported by new releases.



Configuration

`voxl-tflite-server` reads its configuration from `/etc/modalai/voxl-tflite-server.conf`, generated and edited by the `voxl-configure-tflite` tool. Run it with no arguments to get an interactive wizard with presets for every bundled model (plus a “Custom” option that prompts for every parameter), or pass flags to script it:

```
voxl-configure-tflite          # interactive wizard
voxl-configure-tflite --model-path /usr/bin/dnn/yolov11n_float16.tflite \
  --model-arch YOLOV11 --norm-type HARD_DIVISION \
  --label-path /usr/bin/dnn/coco_labels.txt \
  --input-pipe /run/mpa/hires_small_color/ --delegate gpu \
  --output-prefix yolo --skip-frames 0
```

Config file parameters:

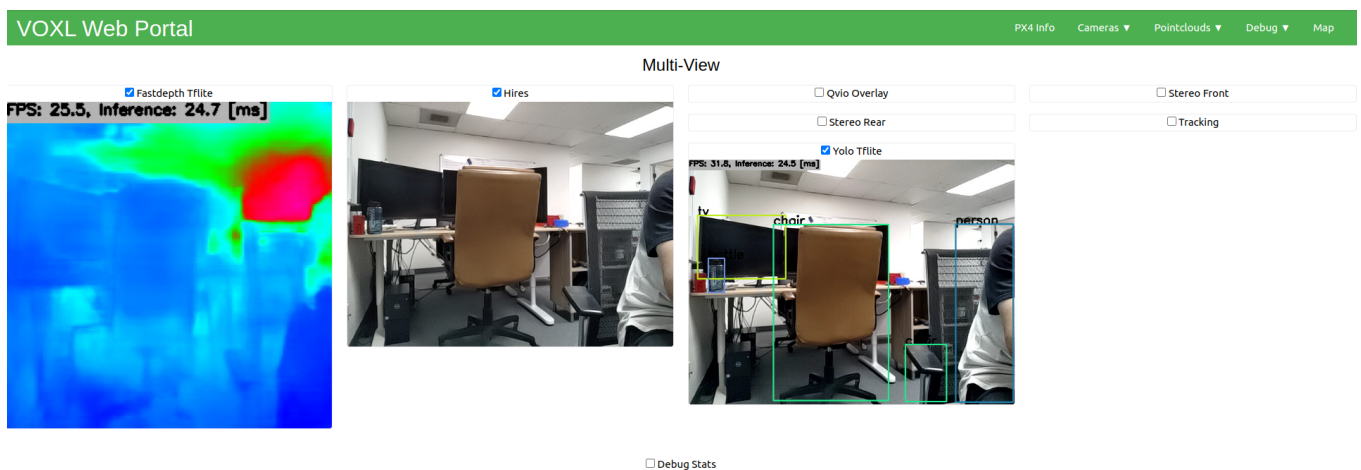
Parameter	Default	Meaning
model	/usr/bin/dnn/...	path to the .tflite model file
model_architecture	MOBILE_NET	how to interpret the model's output tensors. One of MOBILE_NET, MOBILE_NET_CLASSIFIER, YOLOV5, YOLOV8, YOLOV11, EFFICIENT_NET, POSENET, FAST_DEPTH, DEEPLAB. An unrecognized value exits at startup.

Parameter	Default	Meaning
norm_type	PIXEL_MEAN	input normalization: PIXEL_MEAN scales to [-1,1], HARD_DIVISION to [0,1] (used by the YOLO family), NONE leaves [0,255]. A wrong value doesn't error — it silently degrades accuracy.
requires_labels	true	whether the model needs a labels file
labels	/usr/bin/dnn/ coco_labels.txt	path to the labels file
input_pipe	/run/mpa/ hires_small_color/	full path of the camera pipe to consume. The wizard enumerates live camera pipes on your system to choose from.
delegate	gpu	hardware acceleration: gpu (float16 models), nnapi (quantized int8/uint8 models; automatically selects the best accelerator), or cpu (XNNPACK, multi-threaded). If a delegate fails to apply, the server prints an error and falls back to plain CPU.
skip_n_frames	—	frames to skip between inferences. For a 30Hz input, 5 gives ~5Hz output; 0 processes every frame.
allow_multiple	true	allow multiple server instances; output pipes get the output_pipe_prefix prepended
output_pipe_prefix	mobilenet	prefix for this instance's output pipes

Output

With the default configuration (allow_multiple true, prefix mobilenet), the overlay stream is published at /run/mpa/mobilenet_tflite — in general <output_pipe_prefix>_tflite. Set allow_multiple to false to get the un-prefixed names tflite / tflite_data, which some consumers expect. The stream is a normal camera pipe: view it with voxl-portal, convert to ROS with voxl_mpa_to_ros, or log it with voxl-logger.

Each model provides a task-specific overlay, with an fps counter and inference timer in the top left.



Object-detection models additionally publish <output_pipe_prefix>_tflite_data, a metadata pipe with one struct per detection:

```
// struct containing all relevant metadata to a tflite object detection
typedef struct ai_detection_t {
    uint32_t magic_number;
    int64_t timestamp_ns;
    uint32_t class_id;
    int32_t frame_id;
    char class_name[BUF_LEN];
    char cam[BUF_LEN];
    float class_confidence;
    float detection_confidence;
    float x_min;
    float y_min;
    float x_max;
    float y_max;
} __attribute__((packed)) ai_detection_t;
```

The detection pipe also accepts a runtime control command: send set_cam <pipe> to it to switch the input camera on the fly without restarting the server.

Useful flags when running the server by hand: -d (debug output), -t (per-stage timing breakdown), -p <file> (use an alternate config file — this is how multiple instances run), -c (load config and exit).

Benchmarks

Stats were collected across 5000 inferences, using the hires [640x480x3] camera as input, on an older tflite runtime — treat them as relative guidance rather than current absolutes. Current releases pipeline pre-processing, inference, and post-processing on separate threads, so end-to-end throughput is higher than single-pass numbers suggest.

Model	Task	Avg Cpu Inference(ms)	Avg Gpu Inference(ms)	Avg NNAPI Inference(ms)	Max Frames Per Second(fps)	Input Dimensions	Source
MobileNetV2-SSDlite	Object Detection	33.89ms	24.68ms	34.42ms	34.86750349	[1,300,300,3]	link
EfficientNet Lite4	Classifier	115.30ms	24.74ms	16.42ms	48.97159647	[1,300,300,3]	link
FastDepth	Monocular Depth	37.34ms	18.00ms	37.32ms	45.45454546	[1,320,320,3]	link
DeepLabV3	Segmentation	63.03ms	26.81ms	61.77ms	32.45699448	[1,321,321,3]	link
Movenet SinglePose Lightning	Pose Estimation	24.58ms	28.49ms	24.61ms	34.98950315	[1,192,192,3]	link
YoloV5	Object Detection	88.49ms	23.37ms	83.87ms	36.53635367	[1,320,320,3]	link
MobileNetV1-SSD	Object Detection	19.56ms	21.35ms	7.72ms	85.324232082	[1,300,300,3]	link
MobileNetV1	Classifier	19.66ms	6.28ms	3.98ms	125.313283208	[1,224,224,3]	link

Newer YOLOv8 and YOLOv11 models ship with the server (`/usr/bin/dnn/yolov8n_float16.tflite`, `yolov11n_float16.tflite`) and are the recommended starting point for object detection.

Custom Models

You do **not** need to modify or rebuild `voxl-tflite-server` to run your own model, as long as its outputs match one of the supported architectures (`model_architecture` above). The workflow is: convert your model to `.tflite`, copy it to the target, and point the config at it.

1. Convert your model to LiteRT

YOLO (Ultralytics) — export directly to TFLite; no frozen graph is involved (Ultralytics no longer produces one):

```
from ultralytics import YOLO
YOLO("best.pt").export(format="tflite") # produces best_saved_model/best_float16.tflite
```

PYTHON

Under the hood this goes PyTorch → ONNX → onnx2tf (<https://github.com/PINTO0309/onnx2tf>) → TFLite. Our `voxl-train-yolov8` (<https://gitlab.com/voxl-public/support/voxl-train-yolov8>) repo provides a complete training + export environment with working dependency pins (notably `onnx2tf>1.17.5,<=1.22.3`).

TensorFlow models — use the modern converter on a SavedModel or Keras model:

```
import tensorflow as tf
converter = tf.lite.TFLiteConverter.from_saved_model("path/to/saved_model")
converter.target_spec.supported_types = [tf.float16] # float16 for the gpu delegate
tflite_model = converter.convert()
open("model_float16.tflite", "wb").write(tflite_model)
```

PYTHON

Other PyTorch models — export to ONNX first, then convert with onnx2tf (<https://github.com/PINTO0309/onnx2tf>), or use Qualcomm AI Hub (<https://aihub.qualcomm.com/>) with the 'RB5 (Proxy)' device.

Quantization determines which delegate you can use: float16 models for `gpu`, full-integer (int8/uint8) models for `nnapi`.

2. Deploy — no code changes

- Copy your `.tflite` (and labels file, if applicable) to `/usr/bin/dnn/` on target.
- Run `voxl-configure-tflite`, choose **Custom - specify all parameters manually**, and supply the model path, architecture, normalization type, labels, input pipe, and delegate (or pass the equivalent flags shown in Configuration).
- Restart the service: `systemctl restart voxl-tflite-server`.

The input tensor size is read from the model automatically, and camera frames are resized and color-converted for you. YOLO-family models use `norm_type HARD_DIVISION`.

3. Only for genuinely new output layouts — extend the server

If your model's output tensors don't match any supported architecture, add one: a new enum in `include/model_helper/model_info.h`, a mapping in `get_model_type()` in `src/main.cpp`, a case in `create_model_helper()` in `src/model_helper/model_helper.cpp`, and a `ModelHelper` subclass (under `include/model_helper/` and `src/model_helper/`) overriding `post_process()` and `worker()` — `preprocess()` and `run_inference()` have generic base implementations you can usually keep. The existing helpers (YOLOv11 simply reuses the YOLOv8 helper) are good templates. To surface your model as a wizard preset, add an entry in `scripts/voxl-configure-tflite-wizard` and the architecture to the bash completion.

Running Multiple Models

The wizard supports this directly: run `voxl-configure-tflite`, and when asked how many instances to configure, answer 2 or more. It writes one config per instance (`/etc/modalai/voxl-tflite-server_2.conf`, ...) with distinct `output_pipe_prefix` values and updates the systemd service to launch each instance with `-p <its config>`. Each instance publishes under its own prefix (e.g. `/run/mpa/yolo_tflite`, `/run/mpa/depth_tflite`).

NOTE

The board heats up much quicker when running multiple models. Adding an external fan when benchtop testing helps combat the extra heat.

Current Limitations

Limitations of the current implementation:

- **Only the `model_architecture` values listed above are supported.** YOLO support means **v5, v8, and v11** (v11 shares the v8 decoder). A model with any other output layout requires extending the server in C++.
- **Detection thresholds are compile-time constants**, not configurable: SSD/MobileNet score 0.6; YOLOv5 box 0.40 / class 0.20 / NMS 0.50; YOLOv8/v11 score 0.45 / confidence 0.25 / NMS 0.5. Changing them means rebuilding.
- **One model and one camera per server instance.** Run more models with multiple instances (see above). Switching cameras at runtime (`set_cam` on the control pipe) works for object-detection models only.
- **Camera input formats: YUV and RAW8** (stereo variants use the left frame). RGB pipes are not handled without a code change. Frames above 4K are rejected, the input resolution is locked at the first frame (which is itself skipped while building the resize map), and mid-stream resolution changes are not supported.
- **Model input dtype must be float32, int8, or uint8** — and quantized inputs receive raw image bytes with no scale/zero-point handling. In practice: **export custom models as float16 and run the `gpu` delegate**; that is the tested path.
- **Inference pauses when nothing consumes the output pipes** (a power optimization). If you're testing over `adb` with no portal or inspect tool attached, it will look dead — attach a consumer, or run with `-d/-t` which disables the optimization.
- **Delegates fail soft:** if the requested delegate can't apply (or the name is unrecognized), the server quietly runs on CPU — check the startup output. There is no Hexagon delegate; `nnapi` picks its accelerator automatically.
The service is capped at 4 GB RAM by systemd and drops frames under backpressure rather than queueing.
Use the **short CLI flags** (`-d` debug, `-t` timing, `-p <conf>`); the long-form flags are currently miswired.
Supported platform: the QRB5165 family (VOXL 2 / VOXL 2 Mini).

Troubleshooting

For general service debugging see Debugging Services.

Custom model runs but produces no detections. Triage:

- 1 Stop the service and run the server in the foreground: `systemctl stop voxl-tflite-server`, then `voxl-tflite-server -d`.
- 2 If the output repeats `Error in TensorData<float>: should not reach here`, your model's **output tensor dtype doesn't match what the server expects** — this is a model-export problem, not a labels problem. Re-export **unquantized** (float16) following `voxl-train-yolov8` (<https://gitlab.com/voxl-public/support/voxl-train-yolov8>), and use the `gpu` delegate.
- 3 Verify detections flow with `voxl-inspect-detections tflite_data -a` (add your `output_pipe_prefix` if set) and visually in `voxl-portal`. If the overlay stream never appears in portal at all, the server failed at startup — read the foreground output.
- 4 A missing labels file exits the server at startup; a labels file with fewer entries than the model has classes shows numeric class ids instead of names.

The board reboots when the server runs. Almost always **power delivery, not software**: viewing tflite output kicks off real inference, the CPU/GPU load spikes, and a worn power-cable contact browns out the board. Replace the 4-pin power cable between VOXL 2 and its power source (repeated unplugging loosens the contacts — a “wiggle fix” is temporary and makes it worse). A true kernel panic reboots into recovery mode, which distinguishes it from a power reset.

Building from source fails. The `install_build_deps.sh` branch argument must match the SDK branch of the source you checked out — `./install_build_deps.sh qrb5165 sdk-1.4` for the `sdk-1.4` branch, `dev` only for tip-of-tree — and the required `voxl-cross` image version tracks the branch (current master needs `voxl-cross ≥ 4.3`). Mismatched deps produce compile/link errors deep in TensorFlow.

Not supported: custom YOLOv5 training assistance, RTSP streams as server input (MPA camera pipes only), and cropping/resizing inside the server — produce an appropriately-sized stream from `voxl-camera-server` instead (e.g. a small or MISP stream).

Demo

Source

Source code is available on GitLab (<https://gitlab.com/voxl-public/voxl-sdk/services/voxl-tflite-server>)

Frequently Asked Questions (FAQ)

I want to use my Python script to pre/post-process inputs/outputs to the model, how can I do this? The inputs and outputs for the model are both done using `libmodal-pipe` which only supports C/C++ and so there isn't a way to directly interface with Python. However, it's possible to use `libmodal-pipe` to create a very simple wrapper which consumes from a pipe, runs a Python executable, and then passes information along to another pipe. In the `libmodal-pipe` repo check out the examples (<https://gitlab.com/voxl-public/voxl-sdk/core-libs/libmodal-pipe/-/tree/master/examples>) dir for an example of setting up a basic pipe interface on VOXL.

Why is my model predicting less accurately when deployed on VOXL? The most common cause is a wrong `norm_type` — it fails silently and just degrades results; check what normalization your model was trained with. Beyond that, `voxl-tflite-server` does its own pre/post-processing on the image tensors which may differ from your local pipeline, and quantization can reduce accuracy versus the non-quantized original.

Copyright © 2019-2026 ModalAI Inc.

Page last modified: Sep 3 2026 at 02:00 PM.